Exercise 8 - Smart Pointers and OOP-Basics

This weeks exercise sheet will focus on implementing some simple classes, as well as creating some 'smart_pointers' from the 'std-library'.

If you have any questions or problems, please write in the Moodle!

Exercise 1 — Your very own classes

For this Exercise create your own `Classes` inside your project folder.
Try out your code *in `main.cpp`.*
You should at least create one `Rectangle` and `Circle` each and calculate both of their areas and print them to the `console`.

a) Implement a `class` called `Rectangle`. In its `header`, define the `private` fields `width` and `height` with appropriate data types. Write a method `calculate_area`, which does exactly that and returns the solution.
The area of a rectangle is equal to its width multiplied by its height.

b) Implement a `class` called `Circle`. In its `header`, define the `private` field `radius` with an appropriate data type. Write a method `calculate_area`, which does exactly that and returns the solution.
The area of a Circle is equal to its radius squared multiplied by pi (You can use 3.141).

c) Write `getter` and `setter` methods for all fields of both your classes.

Exercise 2 — Smart pointers

For this Exercise try out your code in `main.cpp`.

a) Declare a `std::unique_ptr` holding the integer value `42`. Declare another `std::unique_ptr` and try to assign the first pointer to it. What happens? Explain the occurring behaviour with respect to the idea of a unique pointer.

b) Now use `std::move(<unique pointer>)` to assign the first pointer to the second one. What happened to the first pointer now? Find out with a bit of `trial and error` and explain your results.
You can look up `unique_ptr`s via the lecture slides or by browsing the web.